

EC 503 COURSE PROJECT

“Learning from Data”

Date: February 18, 2026

Deadlines:

Team and topic: February 25, 2026

Proposal: March 20, 2026

Progress report: April 10, 2026

Presentations: April 29 & April 30, 2026

Final report and code: May 1, 2026

The course project is one of the core elements of the learning process in EC503, and a major contributor to the final course grade. Thus, it is very important to devote a sufficient amount of effort and time to do well in the project. The project has four main objectives:

1. to test and extend your knowledge acquired in the course on a contemporary machine learning problem,
2. to test your skills in literature search while correlating the knowledge you acquired in the course with the work of others,
3. to test your ability to map a machine learning problem onto a mathematical formulation, analyze it, and then to map this formulation onto a set of instructions in *Matlab*, *R*, *Python*, *C/C++/C#*, *Java*, or *Weka*,
4. to test your ability to communicate the results in written and oral form.

TEAM SIZE AND COMPOSITION

A balanced team is one where (1) analytical, (2) computational, and (3) presentation skills are adequately represented and where team members work together conscientiously and cheerfully with mutual respect and aim to learn from each other and grow. The team size is 2 – 3 students due to a combination of pedagogical and logistical considerations.

TIMELINE

First, select one or two teammates and a topic according to the guidelines listed in the PROJECT GUIDELINES section of this document. By February 25, 2026, each team submits to Gradescope (as a single group-submission) a document containing a tentative title, names and BU email addresses of all the team members, and an informal 1–2 paragraph description of the topic the team plans to work on including suitable references. Make sure to select all team members when making a group-submission to Gradescope.

The project will be executed in 3 phases as outlined below. Please pay careful attention to the deadlines, which are also listed at the top of this page.

- *Phase I:* Then each team carefully researches the literature using conference and journal papers, books, or the Internet, and prepares and submits a formal 2-page proposal to Gradescope (as a single group-submission) by March 20, 2026. Details about what to include in the proposal are explained in the REPORTS section of this document. It should contain a title, names and BU email addresses of all the team members, and a description

of the topic the team plans to work on including suitable ML methods, datasets, and references. Make sure to select all team members when making a group-submission to Gradescope.

- *Phase II:* Each team further researches the topic, now in more depth, using various resources, analyzes, implements and then evaluates one or more algorithms in *Matlab*, *R*, *Python*, *C/C++/C#*, *Java*, or *Weka* on one or more datasets. You may use native functions in *Matlab*, *R*, *Python*, *etc.*, or those available on the Internet (except for a complete solution to your problem, if it exists). Each team prepares a progress report that extends the original proposal by describing steps completed thus far, preliminary results, and any issues encountered. This report, no more than 4 pages long, is due on April 10, 2026 in Gradescope as a group-submission. See the REPORTS section for details on report content.
- *Phase III:* Based on the completed research, each team prepares for and gives a 10-minute oral presentation. All teams are expected to participate in the oral presentations scheduled on April 29 & April 30, 2026 and answer questions related to their projects (see PRESENTATION section for details). In parallel with getting ready for your presentation, prepare a final report (up to 6 pages; see REPORTS section for details), which is due on May 1, 2026. You are also required to submit the slides of your presentation and the project-code to a specified location (TBD) by May 1, 2026.

PROJECT GRADE BREAKDOWN

5%	Proposal
5%	Progress Report
5%	Software
35%	Presentation
50%	Final Report

It is expected that each team member will participate and contribute to every aspect of the project from problem formulation, literature review, algorithm development, and mathematical analysis, to coding, discussion and comparison of results, report-writing, and the presentation. Different team members may assume leadership roles and do the “heavy lifting” in different aspects of the project. However, everyone is expected to be fully aware of what everyone else is doing. The roles and contributions of each team member should be clearly discussed in the final report, e.g., who wrote which part of the final report, who was responsible for which aspect of coding, who did which part of the literature survey, who analyzed which algorithm, etc.

The final report, presentation, and code will be subjected to a plagiarism check. Failing any of these checks can have a serious consequence on your academic status at BU. If you are not a native speaker of English, please note that reproducing text from the literature verbatim in your report will be treated as plagiarism even if you cite the source and enclose the borrowed text within quotation marks. **You are expected to explain ideas using your own words.** You may take the help of your team members to refine your explanation, but copying from published work is absolutely not permitted.

Although communication between teams is permitted, collaboration is not. All the work should be carried out by each team independently. The remaining classmates and other presentation participants (other faculty, students, etc.) will participate in the evaluation of your project

presentation.

REPORTS

The project proposal, progress, and final reports must be typeset and submitted electronically in pdf or word format. The file must be uploaded to Gradescope as a group-submission and care must be taken to ensure that all members of your team are selected when submitting.

Project proposal

The project proposal should not exceed 2 pages of text (figures, tables, references, and the division of labor section are extra and are not counted towards the page limit). It should describe the problem under investigation, briefly review the literature and outline the proposed work and tasks (including datasets and code you need to write). The description of proposed work should be at two levels: an ideal objective, that the team hopes to achieve, and a fall-back plan, in case the ideal turns out to be unattainable. A list of references should be included at the end of the proposal. The last section must clearly explain the division of labor, i.e., what each member is primarily responsible for, preferably as a bulleted list.

Progress report

The progress report should expand upon the proposal by elaborating on work accomplished, work in progress, and work that needs to be done. This includes experimental results, augmenting the literature review, explaining challenges ahead, changes to the division of labor, etc. It should not exceed 4 pages of text (figures, tables, references, detailed mathematical derivations, and the division of labor section are extra and are not counted towards the page limit).

Final report

You must use one of the report templates, either in Microsoft Word or in L^AT_EX, available in the “Project” section of the course web site. Please do not change the report format. These templates have been taken from the author-kit of a CVPR conference. The final report should be structured as follows:

1. *Introduction*: Introduce the goals of the project (no detail, just overview). Motivate the study and discuss how it is similar to or different from other problems in machine learning. Specifically, place your work within a broad context of the field, discuss connections to other problems, and the importance/relevance of the specific problems you are proposing to study. Identify any novel or challenging aspects of your project.
2. *Literature Review*: Briefly review the literature by discussing related work. Distinguish between works that are only broadly related to the topic of your project and a smaller subset of key papers (this could be just one or two) that are most directly and closely related to your project. Discuss how the key papers are related to your project and highlight similarities and differences.
3. *Problem Formulation and Solution Approaches*: This could span multiple sections or subsections.
 - Describe, in precise mathematical language, the learning and inference problems being studied in detail, and the solution methodologies that you are investigating.

Clearly identify the underlying constraints and assumptions: What is assumed to be known? What is unknown? What is to be learned using what? What is to be predicted using what? Are there any constraints on what you are allowed to do or not do? Describe the underlying probabilistic models (if any).

- Describe the learning and inference algorithms and analyze their properties mathematically. Specifically, discuss the final optimization problems (maximization or minimization of suitable objective functions) associated with the algorithms. Are there any theoretical properties, e.g., unbiasedness, asymptotic consistency, sample complexity, convergence rate, etc., available for the algorithms?
 - Discuss the underlying intuition and key ideas behind the models and algorithms.
 - Discuss any overfitting issues (curse of dimensionality) and the use of regularization to mitigate them.
 - Discuss the time/memory complexities of the algorithms.
 - Use consistent mathematical conventions and notation throughout. Think carefully about what mathematical symbols are necessary for the exposition and what are unnecessary. Every symbol and index that appears in the report must be explained at the location of its first occurrence or very close to it.
4. *Implementation*: Describe how you have implemented the proposed methods. Include detailed *pseudo-code* in the appendix (not counted towards the page limit).
 5. *Experimental Results*: Describe the experiments you have conducted on synthetic and real data and the results you have obtained (datasets, performance measures used, sensitivity to initialization or other algorithm parameters, impact of real-world degradations like missing data, etc.). Discuss the appropriateness of the performance metrics that you have chosen. Discuss if your results are consistent with your understanding, i.e., are they expected or unexpected? Do they make sense?
 6. *Conclusion*: Describe what you have learned from the project and what further improvements are possible.
 7. *Description of Individual Effort*: At the end of the report, please include a brief description of each project member's contribution to the project. In particular, discuss who wrote which part of the final report, who was responsible for which aspect of coding, who did which part of the literature survey, and who analyzed (theoretically or intuitively) which algorithm. Like the appendix, this falls outside the page limit.
 8. *References*: Include the list of references you used (again not counted towards the page limit).
 9. *Appendix*: **Extra** figures, **extra** tables, detailed mathematical derivations, and detailed pseudo-code.

Note: Unlike the proposal and progress reports where figures and tables were not counted towards the page limit, in the final report, all the **key** figures and tables must be included within the 6 page limit. If there are some finer points that you wish to make which require additional figures and tables, you may include them in the Appendix.

Criteria on which the final report will be evaluated:

1. *Organization, neatness, clarity, logic, and flow*: 5 points
2. *Abstract and Introduction*: 5 points
3. *Literature Review*: 5 points
4. *Problem Formulation, Solution Approaches, and Implementation*: 15 points
5. *Experimental Results*: 15 points
6. *Project difficulty*: 5 points. How challenging was the project in terms of mathematical and algorithmic difficulty, implementation difficulty, and the difficulty of datasets.

PRESENTATION

- The presentation, in PowerPoint or PDF, should consist of not more than 13 slides. Depending on the content, more slides are acceptable if many figures or plots are shown. But you **must not exceed** the 10-minute time limit for your presentation. As a rule of thumb, you should have no more than about 1 slide per minute of presentation.
- The 10 minutes of the presentation should be shared approximately equally by all team members. You should use your own laptop (please test your laptop with the projector in advance to avoid delays during presentation).
- Your presentation should be structured similarly to the report and should contain: introduction, related work, problem formulation and solution approaches, description of your implementation, presentation and discussion of experimental results, and conclusions. The discussion of related work can be much shorter than in the final report and you need not present detailed mathematical derivations (unless that is crucial to your project).
- Use bullet lists, figures and diagrams. Do not put a lot of text in any slide. Short phrases are preferred to long complete sentences. Choose the equations that you want to display carefully (do not blindly dump every single equation from the final report into your presentation). Any symbol that you display must be explained at some level otherwise it need not be displayed at all. Aim to provide an intuitive explanation or interpretation of the underlying mathematics.
- Express your ideas clearly. Remember that other students may be less familiar with the topic, so your team must give a clear, well-organized presentation. Please practice a “dry run” of your presentation in front of your friends.

Please note that you will be asked questions, some of them quite detailed, about your work during the oral presentation. Therefore, you must come prepared to the presentation in terms of both theoretical underpinnings of your work as well as your implementation. In addition to the course instructor and your classmates, other faculty and students may be present.

Criteria on which the presentation will be evaluated by the audience:

1. Are the project goals and significance clear?
2. Is the problem formulation clear? (assumptions, constraints, model, data, etc.)
3. Do you understand the solution methodology (algorithms) and the key insights?
4. Are the implementation details and main experimental results clear?
5. Is the flow of the presentation logical and the slides neat, clear and easy to follow?
6. Is this a challenging project? (mathematically, algorithmically, implementation, or datasets)

PROJECT GUIDELINES

- Focus on exploring learning-frameworks and algorithms instead of specific applications or datasets, i.e., do not tie your project down too strongly to a specific dataset or a specific application. Aim to explore and compare (pros and cons of) a few learning methods.
- Try to explore at least one algorithm that is not part of the course syllabus and at least one that is part of the syllabus.
- Try to compare performance according to more than one metric and ideally also report confidence bounds.
- Test algorithms on least 2 datasets: one clean small-scale dataset, which could even be synthetically generated, on which you can quickly develop and debug your code and one larger real-world dataset.
- Choose datasets wisely: do not select datasets that require an enormous amount of pre-processing for selecting good features for learning. Although very important in practice, it is not the focus of this course. Preprocessing can quickly turn into a huge time sink. You want to focus on understanding and comparing different learning algorithms. Focus on *tabular datasets*, i.e., where there are many different types of features. Traditional ML methods are well-suited to such datasets and can even outperform methods based on Deep Learning.
- It would be nice to study the impact of at least one type of real-world challenge on the performance of different algorithms as well as some methods to deal with them. Examples of challenges include unbalanced data, missing feature components, very different dynamic ranges of features, outliers, or the presence of categorical and quantitative features.
- If it makes sense for your problem, it would be nice to explore the value of some kind of regularization to mitigate the impact of high data dimensionality or predictor-complexity relative to the size of the training set.
- It would be nice to have some discussion and comparison of the time/memory complexities of the algorithms.
- The scale of experimental validation, e.g., number of algorithms, number of datasets, size and complexity of datasets, etc., should be commensurate with the size of the team.

TRADITIONAL VERSUS DEEP LEARNING

This is a first, foundational course on Machine Learning (ML) with a focus on Traditional (Classical) ML methods. Although Deep Learning (DL) has emerged as a very powerful approach in recent years, traditional methods are still very useful in many scenarios and are still widely used in a number of applications, especially in medical, financial, business, and social sciences.

While Deep Learning performs well on unstructured audio-visual and text data, they typically require massive amounts of data, memory, compute power, and time to train well. Figuring out the right architecture, optimization method, or other hyperparameters is still an art than a science. Moreover, their behavior and predictions are difficult to understand and interpret. Deep Learning is primarily useful as a principled approach to learn good representations (features) from large, raw, unstructured data, i.e., it automates feature design in a principled way.

When data is “tabular”, i.e., structured or containing many very different types of features, or is small in size, traditional methods work quite well and are conceptually easier to understand or interpret. Interestingly, many Kaggle competitions are not won by Deep Learning methods. Moreover, even where Deep Learning is used, traditional methods prove useful in the last prediction layer of the Deep Neural Network.

If there are some members in your team who are well-versed with using Deep Learning tools, then you could try to compare the performance of traditional ML methods explored in your project with a DL method.

Some references: Tabular Data: Deep Learning is Not All You Need <https://arxiv.org/abs/2106.03253>; Do We Really Need Deep Learning Models for Time-Series Forecasting? <https://arxiv.org/abs/2101.02118>; Deep Neural Networks and Tabular Data: A Survey <https://arxiv.org/abs/2110.01889>.

PROJECT IDEAS

You are encouraged to propose your own project topics, but here are some ideas you could consider exploring to get you started:

- **Dimensionality reduction for joint reconstruction and label prediction:** Unsupervised methods for dimensionality reduction may not always find best low-dimensional representations for downstream supervised tasks. On the other hand classifiers or regression functions reduce data to scalar predicted labels from which it is not possible to recover data. Explore methods to do both jointly. A reference: Supervised dimensionality reduction for big data <https://doi.org/10.1038/s41467-021-23102-2>.
- **Feature Selection Methods:** A special type of dimensionality reduction is feature selection where only a small subset of a large number of features are retained. Explore a variety of unsupervised and supervised methods to do this and compare them in terms of some supervised downstream task or other performance metrics. Examples of methods include LASSO, group-LASSO, Elastic Nets, Recursive Feature Elimination or Selection, and others.
- **Dimensionality Reduction for Data Visualization:** Learn about and use a variety of methods to visualize high-dimensional data in 2- or 3-D space using principled dimension

reduction techniques such as kernel-PCA, ISOMAP, tSNE, UMAP, etc. Some references: tSNE <https://lvdmaaten.github.io/tsne/>; UMAP <https://arxiv.org/abs/1802.03426>.

- **Time-Series Forecasting:** Explore and compare different statistical and ML methods for predicting univariate or multivariate time-series data. Some references: Do We Really Need Deep Learning Models for Time-Series Forecasting? <https://arxiv.org/abs/2101.02118>; Statistical and Machine Learning forecasting methods: Concerns and ways forward <https://doi.org/10.1371/journal.pone.0194889>; Datasets and competitions: <https://forecasters.org/resources/time-series-data/>.
- **Tackling missing data, outliers, or imbalanced datasets in ML:** Explore methods to address these real-world challenges. Some references: <https://doi.org/10.1371/journal.pone.0262131>, <https://doi.org/10.1007/s10115-022-01772-8>, <https://doi.org/10.1038/s41598-023-29549-1>.
- **Multi-label classification:** Explore methods to predict multiple statistically dependent labels for the same feature vector and how the strength of dependence impacts ability to learn and predict effectively. A recent reference: <https://doi.org/10.1016/j.eswa.2022.117215>.
- ☕ **Fast Gradient Boosted Decision Tree for Multioutput Problems:** <https://arxiv.org/abs/2211.12858>.
- ☕ **Interpretable Decision Trees and Random Forests:** Fast, Interpretable Greedy Tree-Sums <https://arxiv.org/abs/2201.11931>; Hierarchical Shrinkage: improving the accuracy and interpretability of tree-based methods <https://arxiv.org/abs/2202.00858>; MDI+: A Flexible Random Forest-Based Feature Importance Framework <https://arxiv.org/pdf/2307.01932.pdf>.

Main machine learning conferences

- JMLR Workshop and Conference Proceedings: <https://www.jmlr.org/>
- NeurIPS: <https://nips.cc/>
NeurIPS papers: <http://papers.nips.cc/>
- ICML: <https://icml.cc/>
ICML papers (pre-2015): <http://www.machinelearning.org/icml.html>
- AISTats: <https://aistats.org/>
- KDD conferences: <https://kdd.org/>

DATASET SOURCES (incomplete list)

- Google's Dataset Search: <https://toolbox.google.com/datasetsearch>
- UCI Machine Learning Repository: <http://archive.ics.uci.edu/ml/>

- Kaggle: www.kaggle.com
- Medical Information Mart for Intensive Care <https://mimic.mit.edu/>.
- MNIST dataset (unaligned digits different from HW2); Fashion MNIST dataset.
- Human Activity Recognition Dataset.
- MIT-BIH Arrhythmia Database.
- Zillow Housing Dataset (many features; may need regularization).
- Medical Cost Personal Dataset.
- WHO Life-Expectancy Dataset.